Module 7: TypeScript with Frontend Frameworks (React & Angular)

1. Introduction to TypeScript with Frontend Frameworks

TypeScript enhances frontend frameworks like React and Angular by providing type safety, better tooling, and improved code maintainability.

2. Using TypeScript with React

Setting up a TypeScript React project:

Run the following command to create a new React app with TypeScript:

```
sh
```

```
npx create-react-app my-app --template typescript
cd my-app
npm start
```

Defining Props and State in a Component

```
tsx
```

```
import React, { useState } from "react";

interface UserProps {
name: string;
age: number;
}

const User: React.FC<UserProps> = ({ name, age }) => {
const [count, setCount] = useState<number>(0);

return (
<div>
<h1>{name}, Age: {age}</h1>
<p>Counter: {count}</p>
<button onClick={() => setCount(count + 1)}>Increment</button>
</div>
);
};
```

```
export default User;
```

Using TypeScript with React Hooks

```
tsx
const useFetchData = (url: string): [any, boolean] => {
const [data, setData] = useState<any>(null);
const [loading, setLoading] = useState<boolean>(true);

useEffect(() => {
fetch(url)
.then(response => response.json())
.then(json => {
setData(json);
setLoading(false);
});
}, [url]);

return [data, loading];
};
```

3. Using TypeScript with Angular

Setting up an Angular project with TypeScript

Run the following command to create a new Angular project:

```
sh
ng new my-angular-app
cd my-angular-app
ng serve
```

Defining Components with TypeScript

```
typescript
import { Component } from '@angular/core';

@Component({
```

```
selector: 'app-user',
template: `<h1>{{ user.name }}, Age: {{ user.age }}</h1>`,
})
export class UserComponent {
user = { name: 'Alice', age: 25 };
}
```

Using Interfaces in Angular

```
typescript
export interface Product {
id: number;
name: string;
price: number;
}

const product: Product = { id: 1, name: 'Laptop', price: 999 };
```

Angular Services with TypeScript

```
typescript
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
providedIn: 'root'
})
export class DataService {
constructor(private http: HttpClient) {}

fetchData(): Observable<any> {
return this.http.get('https://jsonplaceholder.typicode.com/posts');
}
}
```

4. Best Practices for TypeScript in React and Angular

- Always define types explicitly for props, state, and variables.
- Use `interface` or `type` to define structured data.
- Use `useState<Type>()` in React.
- Avoid using `any` unless necessary.
- Utilize TypeScript for API response validation.

Practice Questions:

1. Create a React component with TypeScript that displays user details.
2. Define an Angular service that fetches data from an API.
3. Use TypeScript interfaces to model a Product object in React.
4. Implement TypeScript in a React component using hooks.
5. Create an Angular component that accepts user input and displays it.

Practice Answers:

1. React User Component:

```tsx
interface UserProps {
name: string;
age: number;
}

const User: React.FC<UserProps> = ({ name, age }) => (
<div>
<h1>{name}, Age: {age}</h1>
</div>
);
```

2. Angular Service:

```typescript
@Injectable({ providedIn: 'root' })
export class ApiService {
constructor(private http: HttpClient) {}

getData(): Observable<any> {
return this.http.get('https://jsonplaceholder.typicode.com/posts');
}
```

```
}
```

3. Product Interface:

```
typescript
interface Product {
id: number;
name: string;
price: number;
}

const product: Product = { id: 1, name: 'Phone', price: 699 };
```

4. React Hook Example:

```
tsx
const [count, setCount] = useState<number>(0);
```

5. Angular Component with Input:

```
typescript
@Component({
selector: 'app-input',
template: `<input [(ngModel)]="name" /><p>Hello {{ name }}</p>`,
})
export class InputComponent {
name: string = '';
}
```